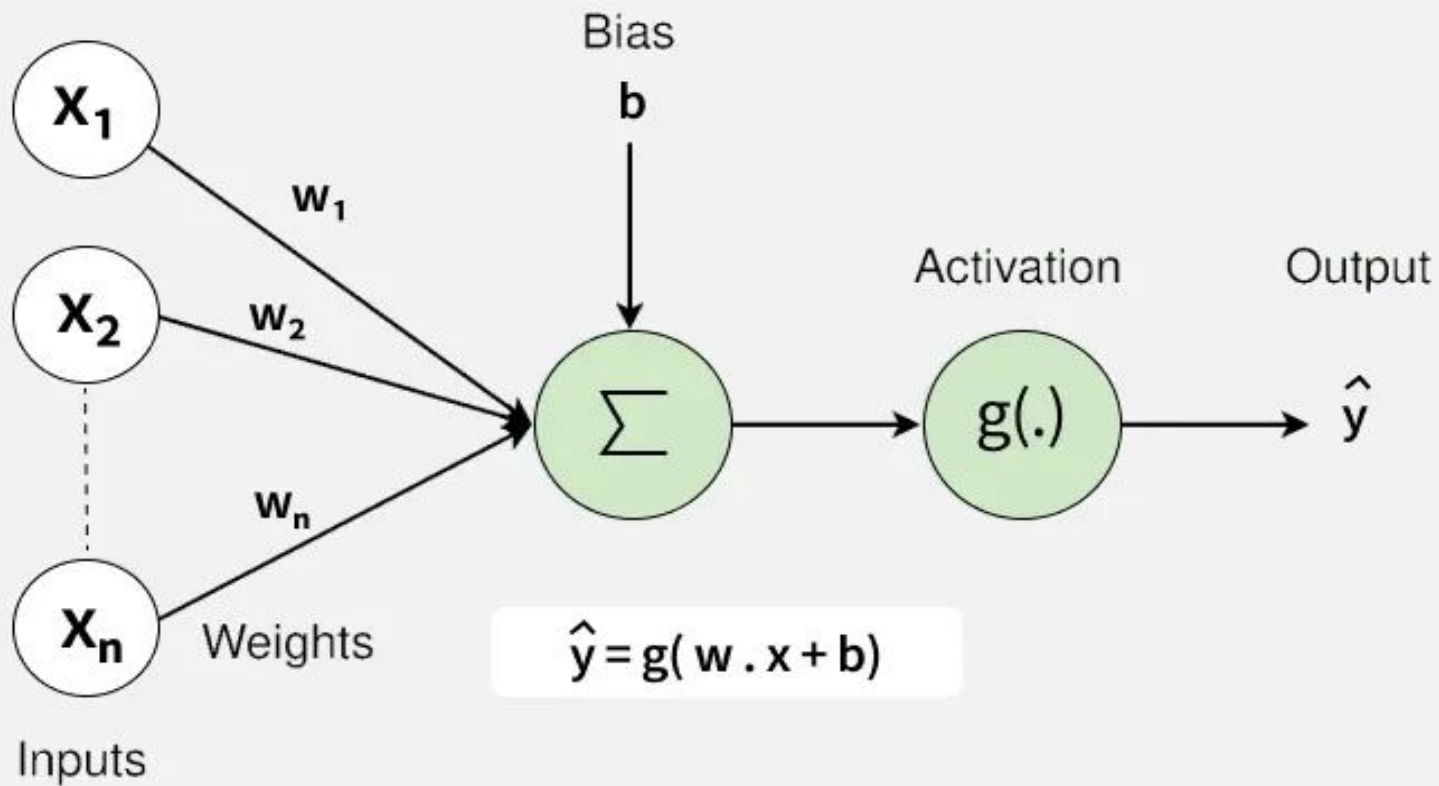
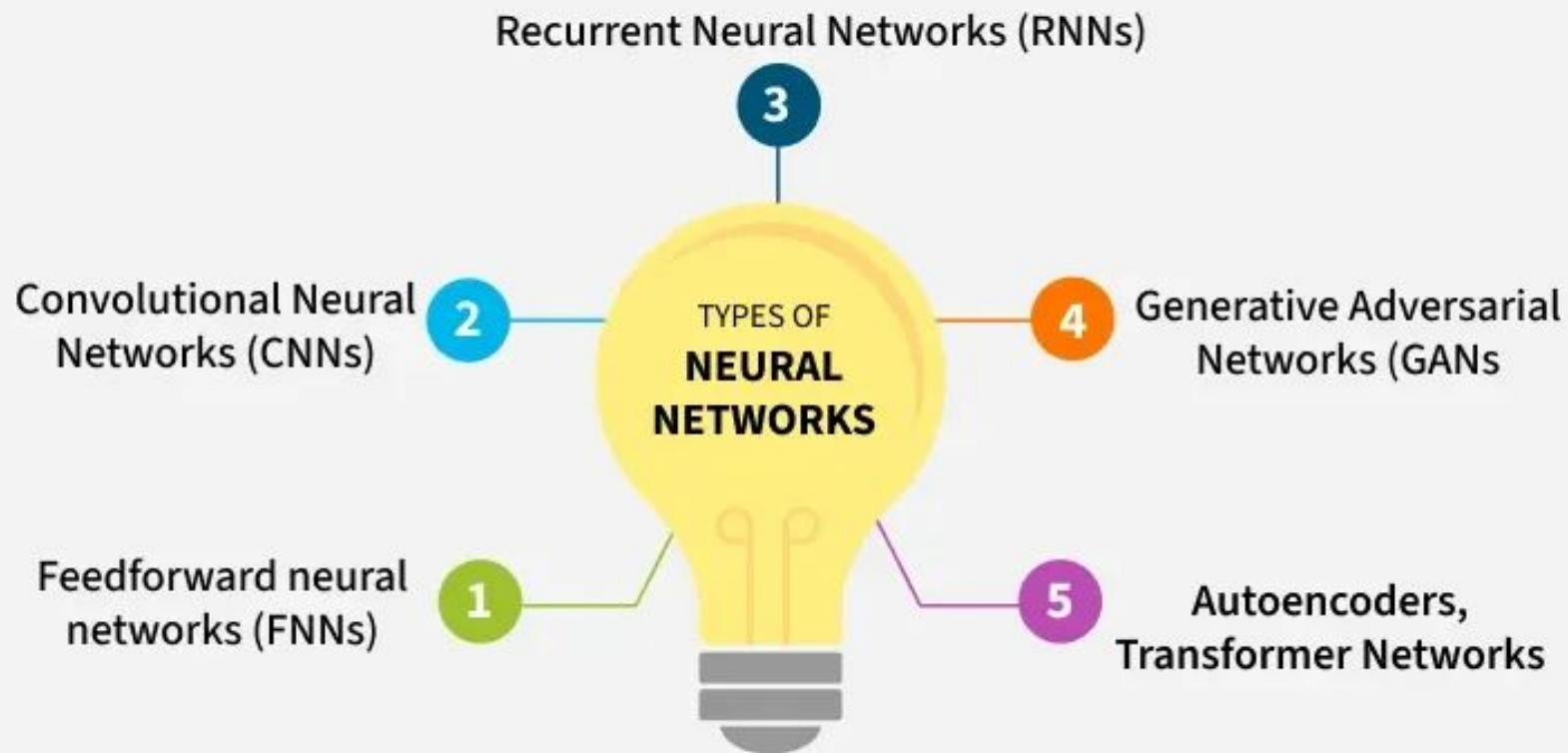
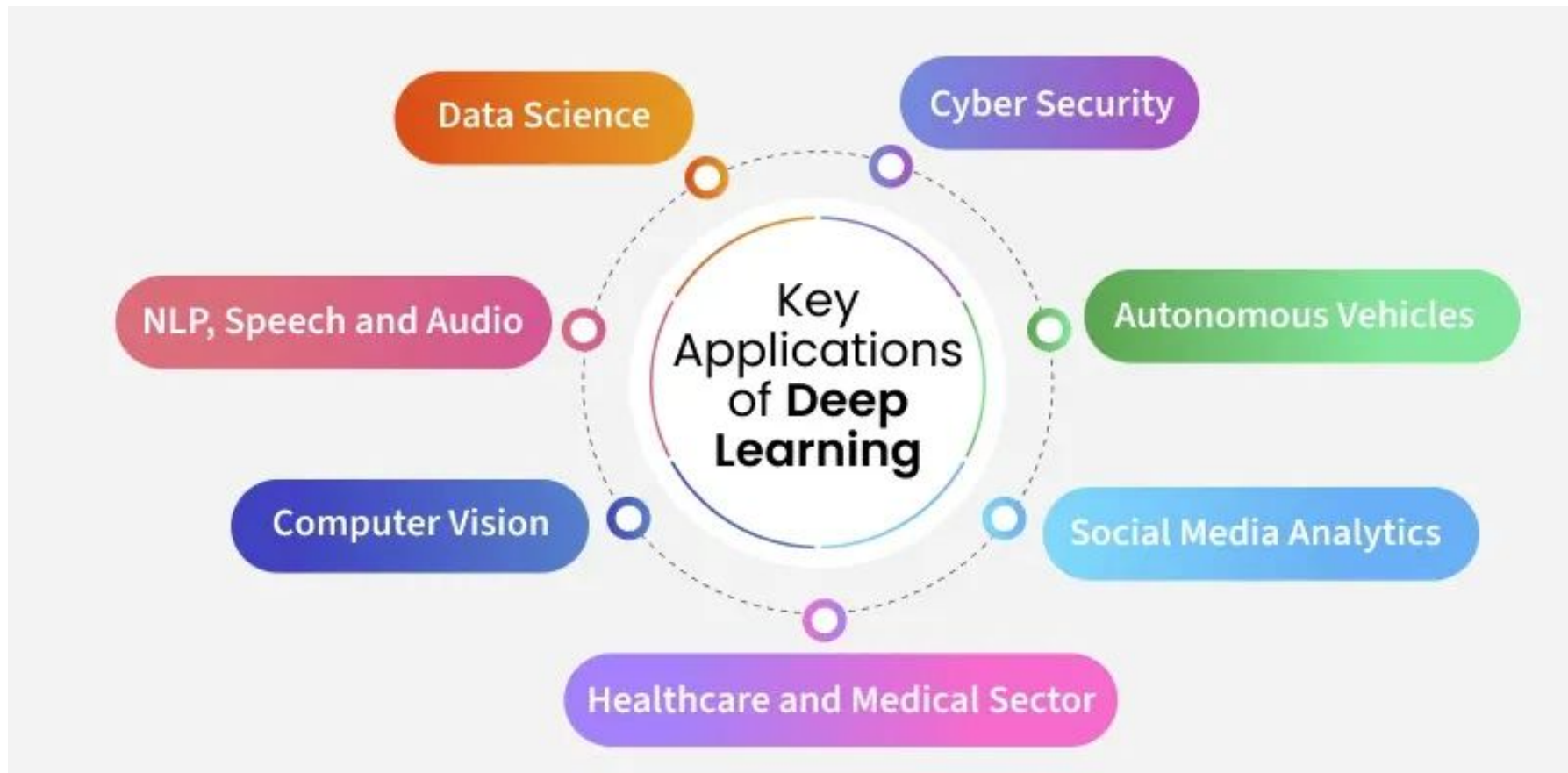
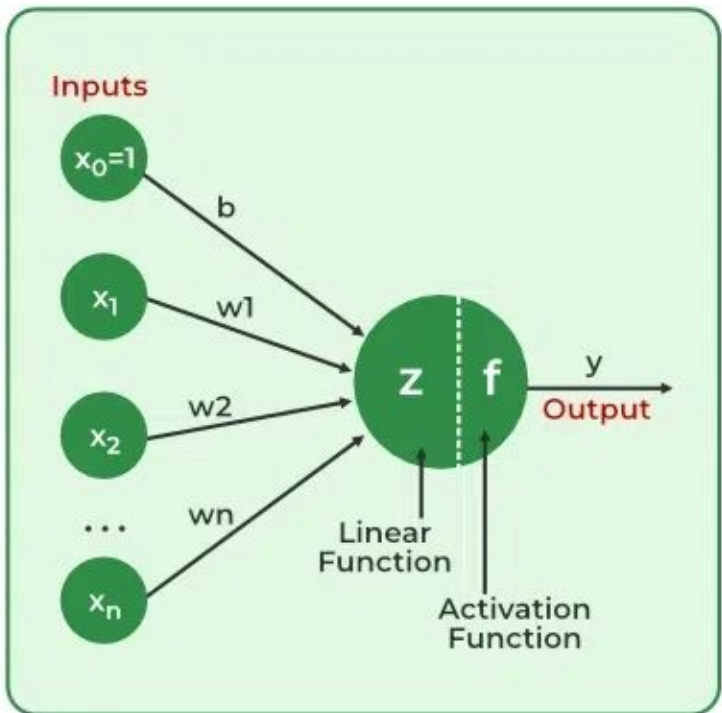
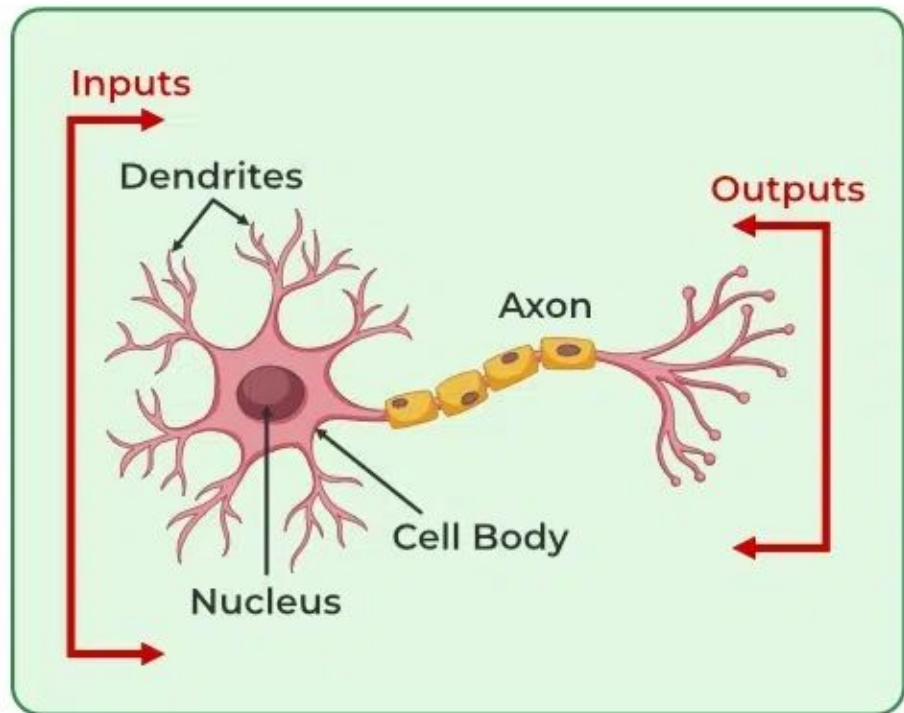


Neural Network









Feedforward Neural Network

Feedforward Neural Network (FNN) is a type of artificial neural network in which information flows in a single direction from the input layer through hidden layers to the output layer without loops or feedback.

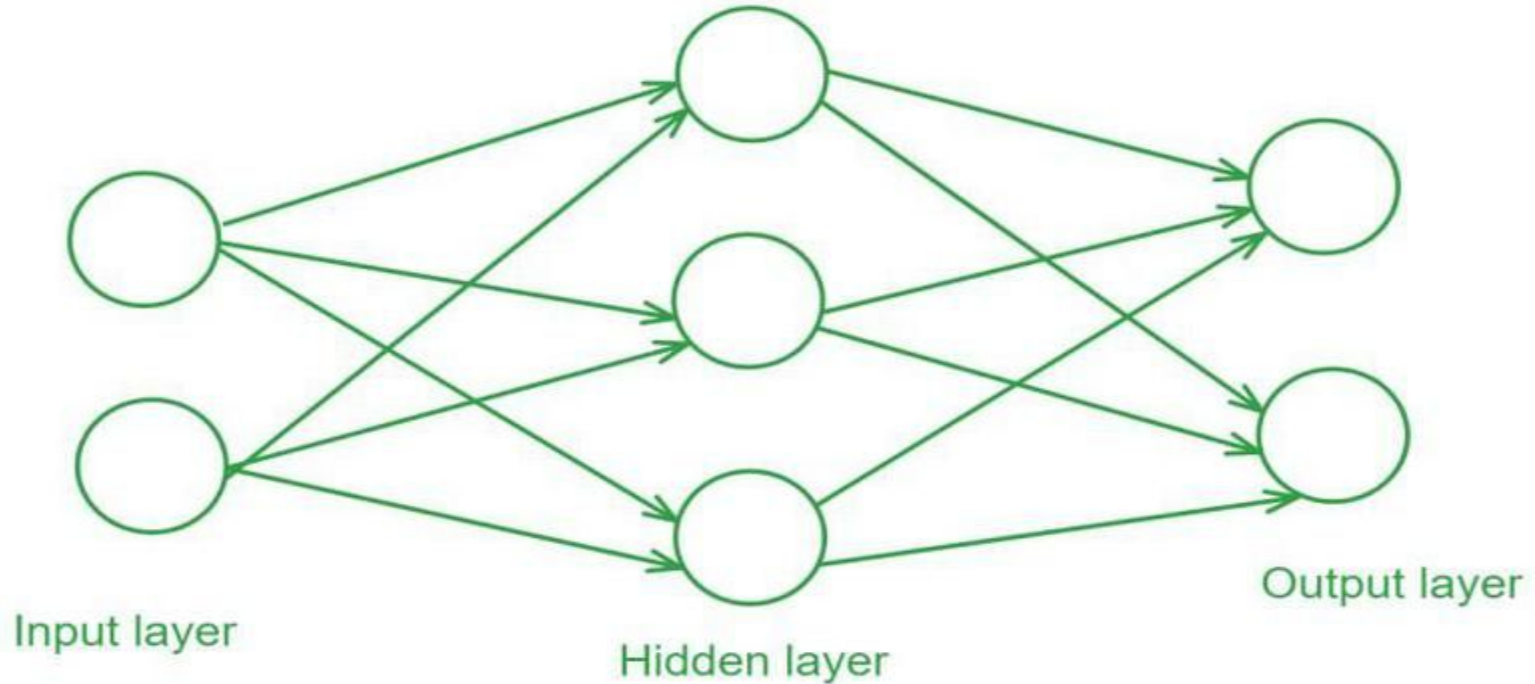
It is mainly used for pattern recognition tasks like image and speech classification.

Structure of a Feedforward Neural Network

Feedforward Neural Networks have a structured layered design where data flows sequentially through each layer.

1. **Input Layer:** The input layer consists of neurons that receive the input data. Each neuron in the input layer represents a feature of the input data.
2. **Hidden Layers:** One or more hidden layers are placed between the input and output layers. These layers are responsible for learning the complex patterns in the data. Each neuron in a hidden layer applies a weighted sum of inputs followed by a non-linear activation function.
3. **Output Layer:** The output layer provides the final output of the network. The number of neurons in this layer corresponds to the number of classes in a classification problem or the number of outputs in a regression problem.

Each connection between neurons in these layers has an associated weight that is adjusted during the training process to minimize the error in predictions.

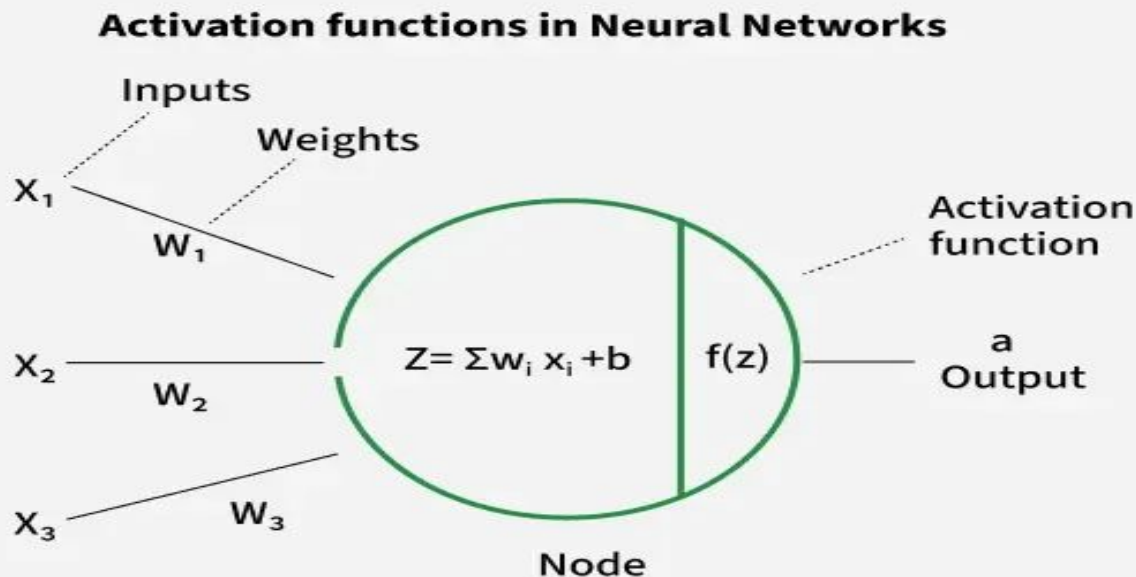


Activation Functions:

- While building a neural network, one key decision is selecting the Activation Function for both the hidden layer and the output layer.
- It is a mathematical function applied to the output of a neuron. It introduces non-linearity into the model, allowing the network to learn and represent complex patterns in the data.
- Without this non-linearity feature a neural network would behave like a linear regression model no matter how many layers it has.

Activation function decides whether a neuron should be activated by calculating the weighted sum of inputs and adding a bias term.

This helps the model make complex decisions and predictions by introducing non-linearities to the output of each neuron.



Introducing Non-Linearity in Neural Network

Non-linearity means that the relationship between input and output is not a straight line. In simple terms the output does not change proportionally with the input. A common choice is the ReLU function defined as

$$\sigma(x) = \max(0, x)$$

$$\sigma(x) = \max(0, x).$$

Imagine you want to classify apples and bananas based on their shape and color.

- If we use a linear function it can only separate them using a straight line.
- But real-world data is often more complex like overlapping colors, different lighting, etc.
- By adding a non-linear activation function like ReLU, Sigmoid or Tanh the network can create curved decision boundaries to separate them correctly.

Effect of Non-Linearity

to introduce a non-linear decision boundary in the input space.

This non-linearity enables the network to learn more complex patterns that are not possible with a purely linear model such as:

- Modeling functions that are not linearly separable.
- Increasing the capacity of the network to form multiple decision boundaries based on the combination of weights and biases.

Without non-linearity even deep networks would be limited to solving only simple, linearly separable problems.

Activation functions help neural networks to model highly complex data distributions and solve advanced deep learning tasks.

Adding non-linear activation functions introduce flexibility and enable the network to learn more complex and abstract patterns from data.

Types of Activation Functions in Deep Learning

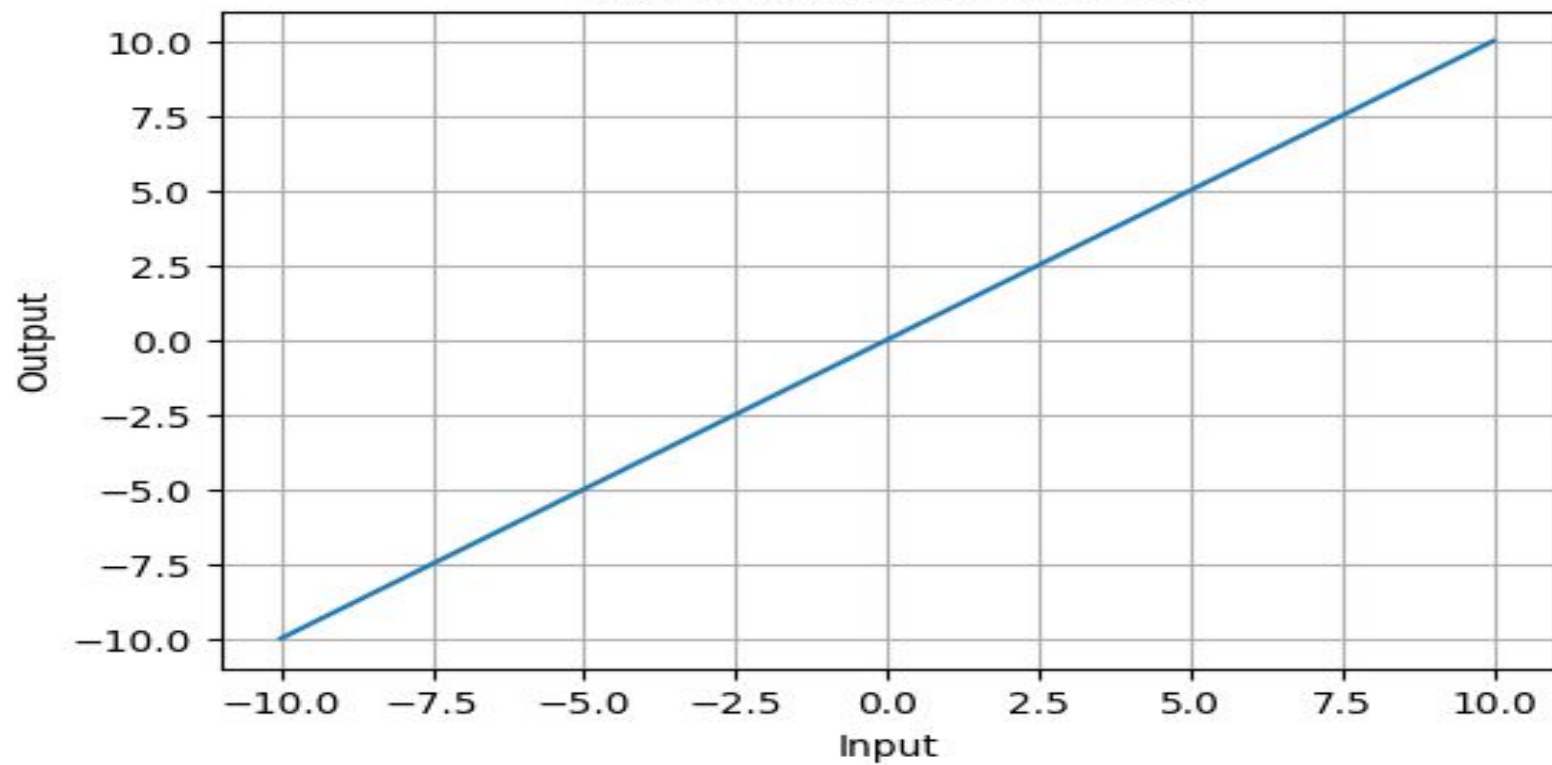
1. Linear Activation Function

Linear Activation Function resembles straight line define by $y=x$. No matter how many layers the neural network contains if they all use linear activation functions the output is a linear combination of the input.

- The range of the output spans from $(-\infty \text{ to } +\infty)$
- Linear activation function is used at just one place output layer.
- Using linear activation across all layers makes the network's ability to learn complex patterns limited.

Linear activation functions are useful for specific tasks but must be combined with non-linear functions to enhance the neural network's learning and predictive capabilities.

Linear Activation Function



2. Non-Linear Activation Functions

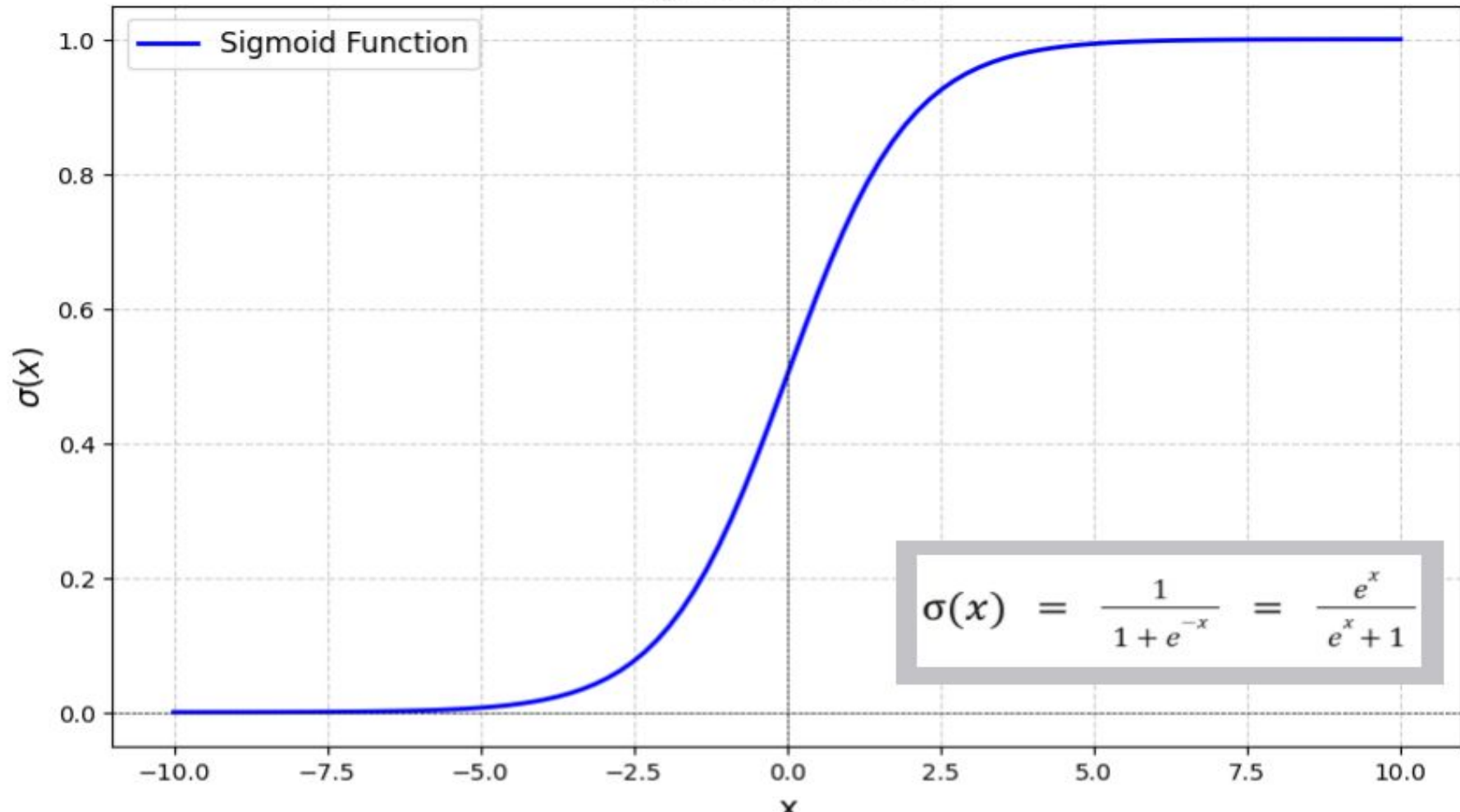
1. Sigmoid Function

Sigmoid Activation Function is characterized by 'S' shape. It is mathematically defined as

This formula ensures a smooth and continuous output that is essential for gradient-based optimization methods.

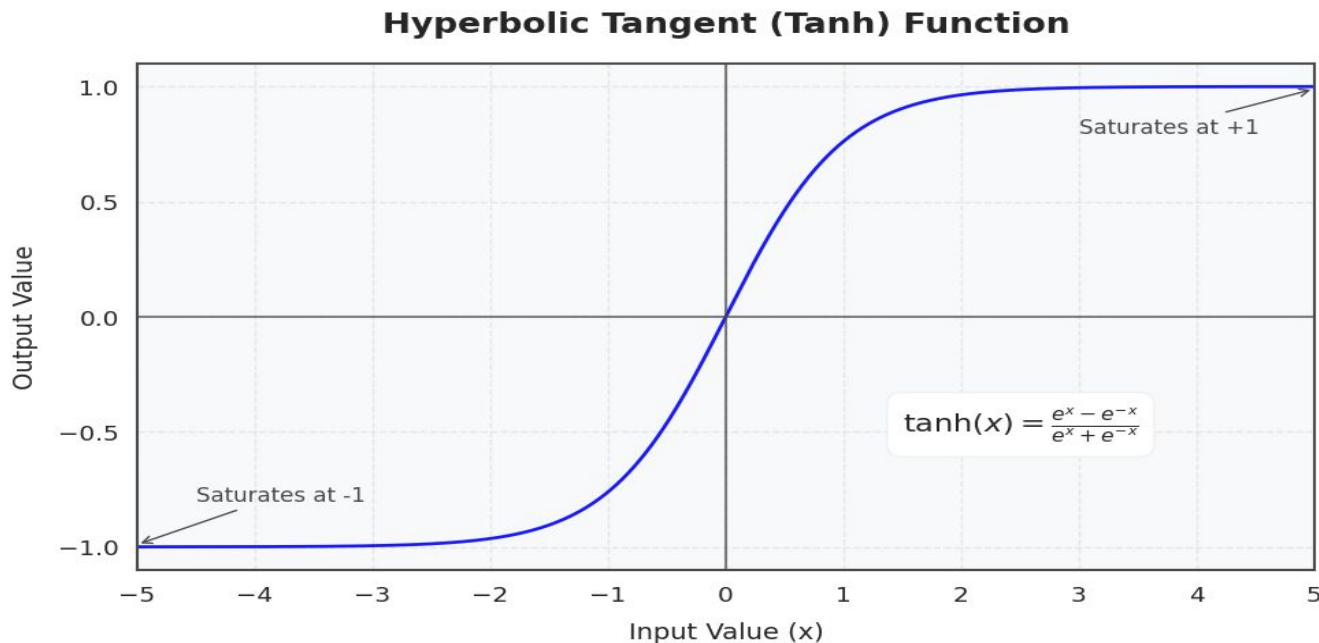
- It allows neural networks to handle and model complex patterns that linear equations cannot.
- The output ranges between 0 and 1, hence useful for binary classification.
- The function exhibits a steep gradient when x values are between -2 and 2. This sensitivity means that small changes in input x can cause significant changes in output y which is critical during the training process.

Sigmoid Function



2. Tanh Activation Function

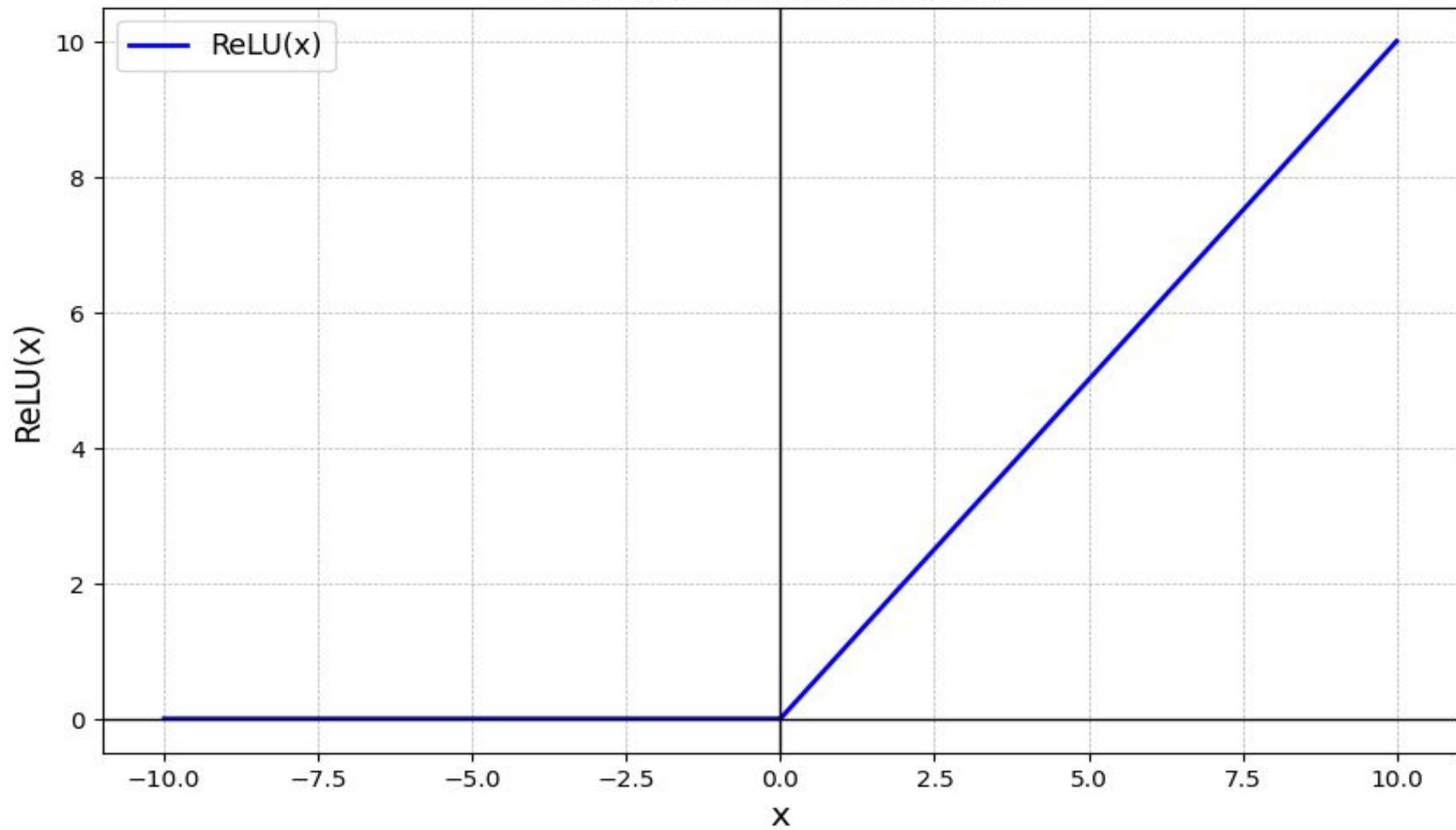
- **Value Range:** Outputs values from -1 to +1.
- **Non-linear:** Enables modeling of complex data patterns.
- **Use in Hidden Layers:** Commonly used in hidden layers due to its zero-centered output, facilitating easier learning for subsequent layers.



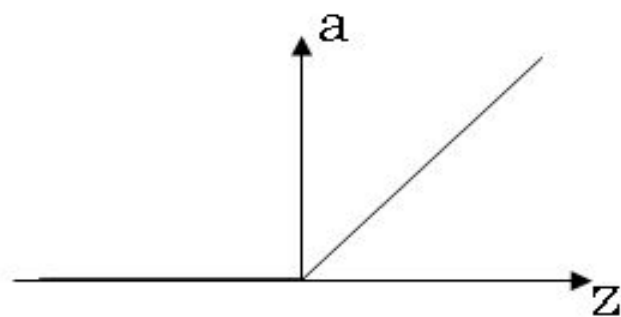
3. ReLU (Rectified Linear Unit) Function

- **Value Range:** $[0, \infty]$ meaning the function only outputs non-negative values.
- **Nature:** It is a non-linear activation function, allowing neural networks to learn complex patterns and making backpropagation more efficient.
- **Advantage over other Activation:** ReLU is less computationally expensive than tanh and sigmoid because it involves simpler mathematical operations. At a time only a few neurons are activated making the network sparse making it efficient and easy for computation.

ReLU Activation Function



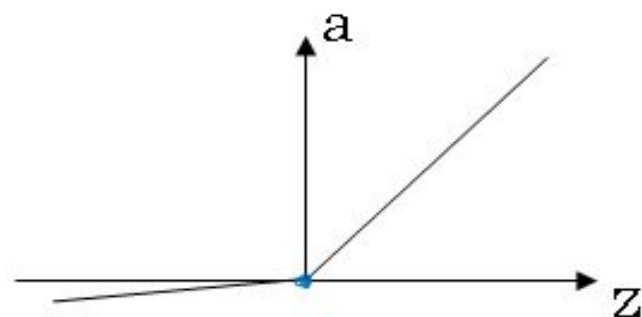
ReLU and Leaky ReLU



ReLU

$$g(z) = \max(0, z)$$

$$\begin{aligned} g'(z) &= 0 \text{ if } z < 0 \\ &= 1 \text{ if } z > 0 \\ &= \text{undefined at } z = 0 \end{aligned}$$



Leaky ReLU

$$g(z) = \max(0.01z, z)$$

$$\begin{aligned} g'(z) &= 0.01 \text{ if } z < 0 \\ &= 1 \text{ if } z > 0 \\ &= \text{undefined at } z = 0 \end{aligned}$$

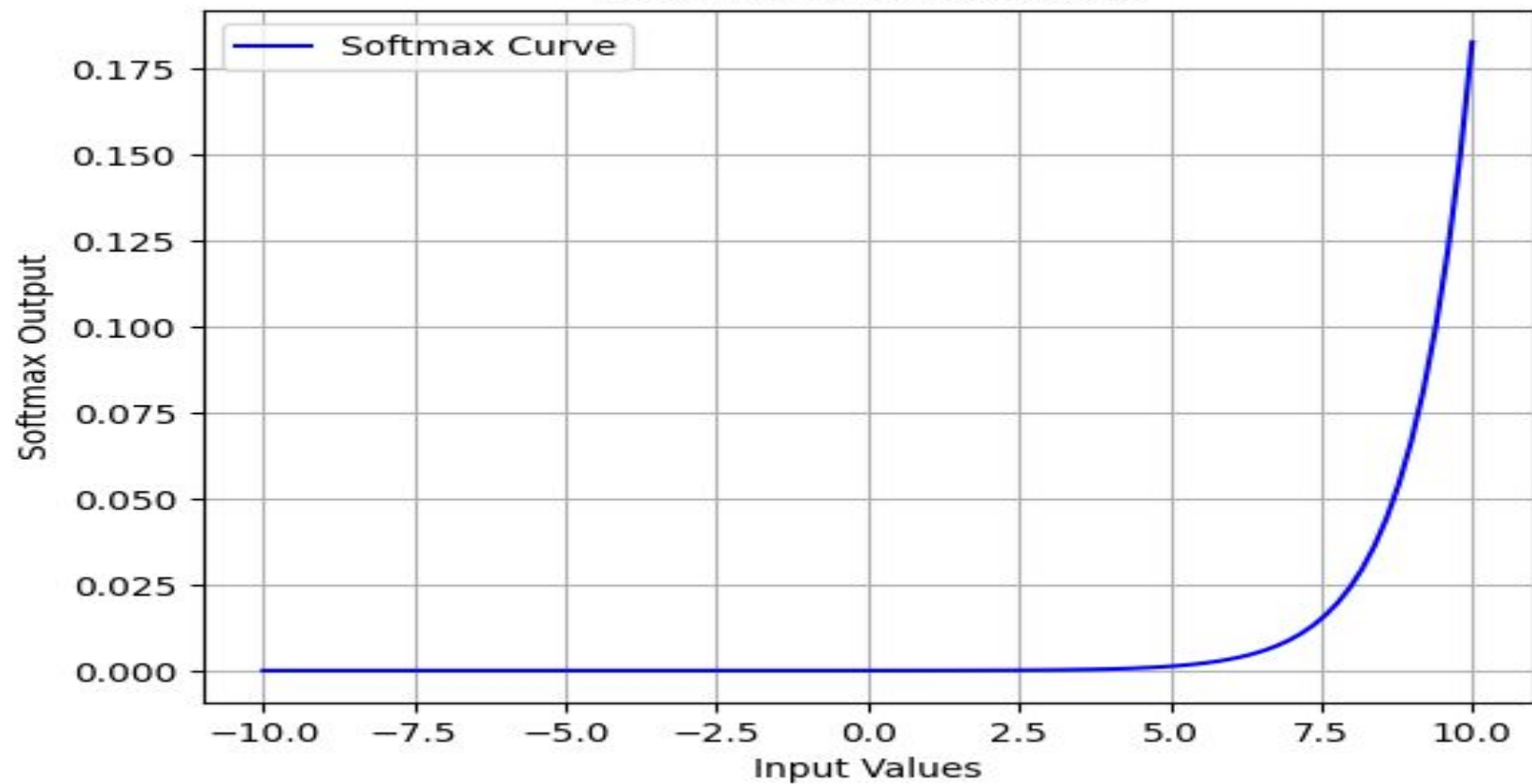
3. Exponential Linear Units

1. Softmax Function

Softmax function is designed to handle multi-class classification problems. It transforms raw output scores from a neural network into probabilities. It works by squashing the output values of each class into the range of 0 to 1 while ensuring that the sum of all probabilities equals 1.

- Softmax is a non-linear activation function.
- The Softmax function ensures that each class is assigned a probability, helping to identify which class the input belongs to.

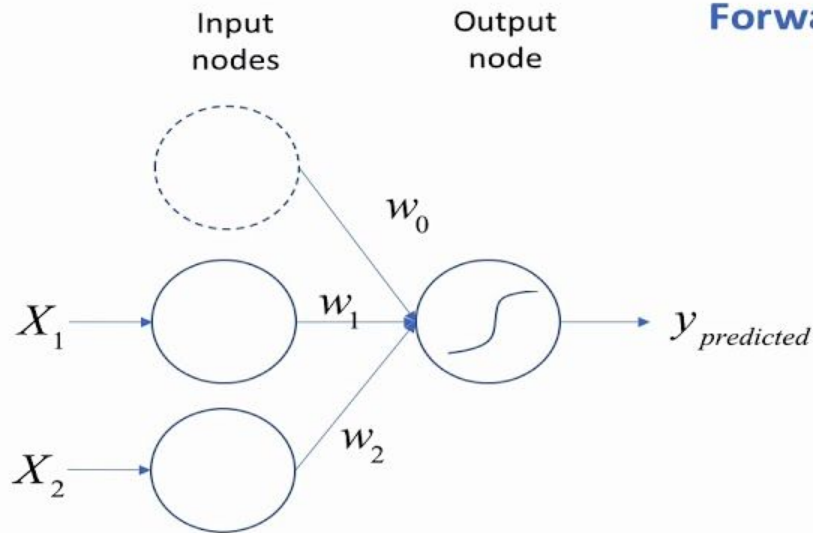
Softmax Function Curve



Training a Feedforward Neural Network

Training a Feedforward Neural Network involves adjusting the weights of the neurons to minimize the error between the predicted output and the actual output. This process is typically performed using backpropagation and gradient descent.

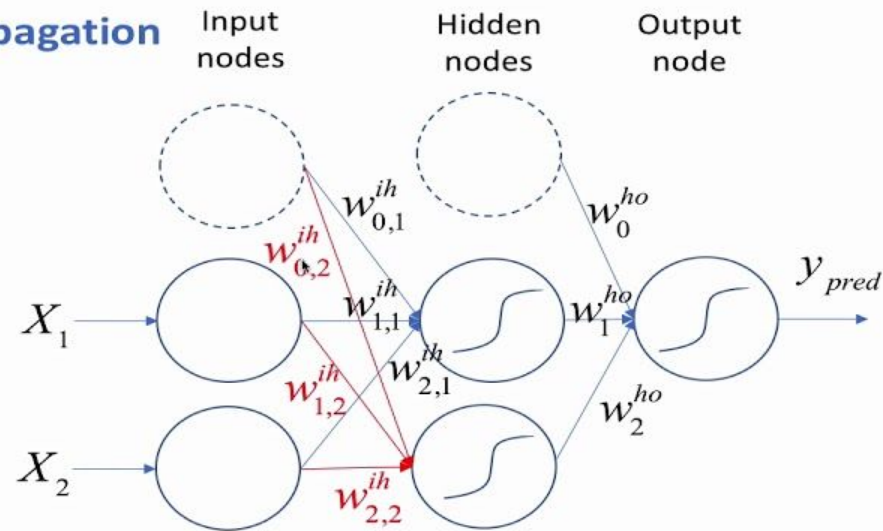
1. **Forward Propagation:** During forward propagation the input data passes through the network and the output is calculated.
2. **Loss Calculation:** The loss (or error) is calculated using a loss function such as Mean Squared Error (MSE) for regression tasks or Cross-Entropy Loss for classification tasks.
3. **Backpropagation:** In backpropagation the error is propagated back through the network to update the weights. The gradient of the loss function with respect to each weight is calculated and the weights are adjusted using gradient descent.



$$X_{hidden} = w_0 + w_1 X_1 + w_2 X_2$$

$$y_{predicted} = act(X_{hidden})$$

Forward Propagation



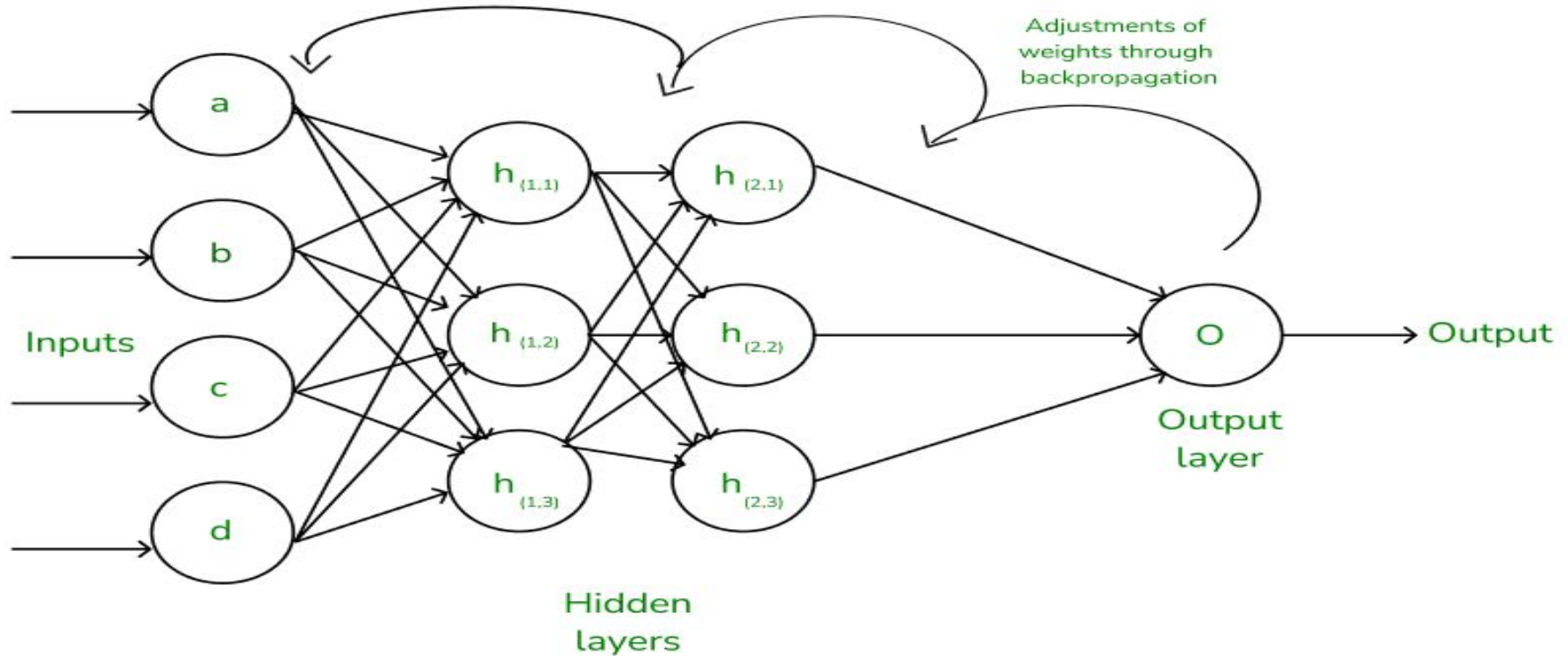
$$X_k^{ih} = w_{0,k}^{ih} + w_{1,k}^{ih} X_1 + w_{2,k}^{ih} X_2$$

$$y_{hidden} = act(\mathbf{X}^{ih})$$

$$X^{ho} = w_0^{ho} + w_1^{ho} y_1^{hidden} + w_2^{ho} y_2^{hidden}$$

$$y_{pred} = act(X^{ih})$$

Backpropagation:



- Back Propagation is also known as "Backward Propagation of Errors" is a method used to train neural network .
- Its goal is to reduce the difference between the model's predicted output and the actual output by adjusting the weights and biases in the network.
- It works iteratively to adjust weights and bias to minimize the cost function.
- In each epoch the model adapts these parameters by reducing loss by following the error gradient.
- It often uses optimization algorithms like **gradient descent** or **stochastic gradient descent**.
- The algorithm computes the gradient using the chain rule from calculus allowing it to effectively navigate complex layers in the neural network to minimize the cost function

Working of Back Propagation Algorithm:

1. Forward Pass Work

In forward pass the input data is fed into the input layer.

These inputs combined with their respective weights are passed to hidden layers. For example in a network with two hidden layers (h1 and h2) the output from h1 serves as the input to h2.

Before applying an activation function, a bias is added to the weighted inputs.

Each hidden layer computes the weighted sum (a) of the inputs then applies an activation function like ReLU (Rectified Linear Unit) to obtain the output (o).

The output is passed to the next layer where an activation function such as softmax converts the weighted outputs into probabilities for classification.

2. Backward Pass

In the backward pass the error (the difference between the predicted and actual output) is propagated back through the network to adjust the weights and bias

Once the error is calculated the network adjusts weights using **gradients** which are computed with the chain rule.

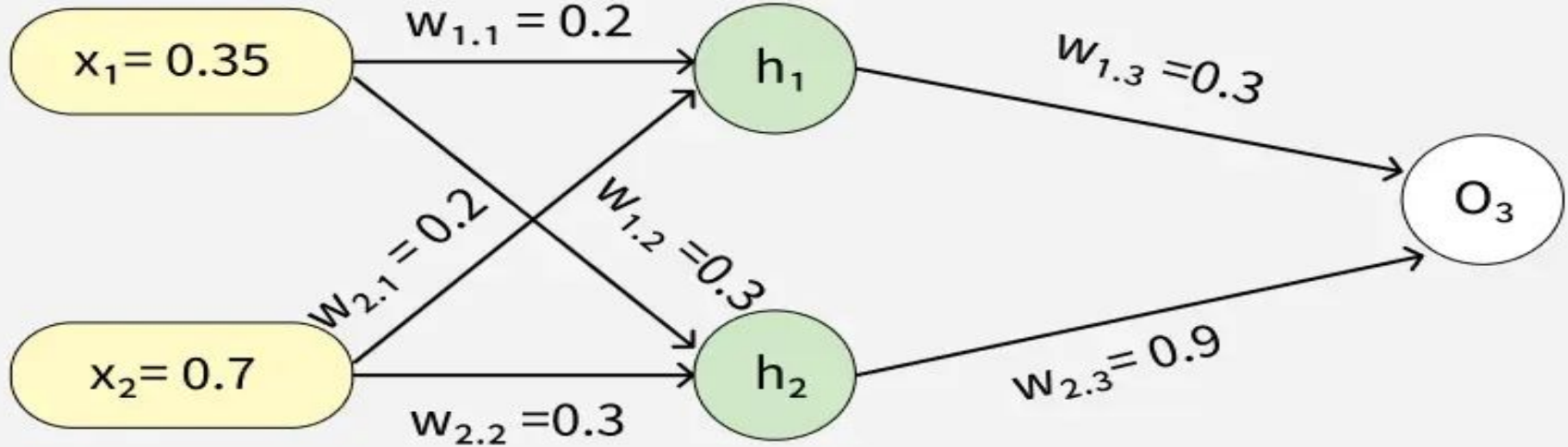
These gradients indicate how much each weight and bias should be adjusted to minimize the error in the next iteration.

The backward pass continues layer by layer ensuring that the network learns and improves its performance. The activation function through its derivative plays a crucial role in computing these gradients during Back Propagation.

Example of Back Propagation in Machine Learning

Assume the neurons use the sigmoid activation function for the forward and backward pass.

The target output is 0.5 and the learning rate is 1.



Forward Propagation

1. Initial Calculation

The weighted sum at each node is calculated using:

$$a_j = \sum (w_{i,j} * x_i)$$

Where,

- a_j is the weighted sum of all the inputs and weights at each node
- $w_{i,j}$ represents the weights between the i^{th} input and the j^{th} neuron
- x_i represents the value of the i^{th} input

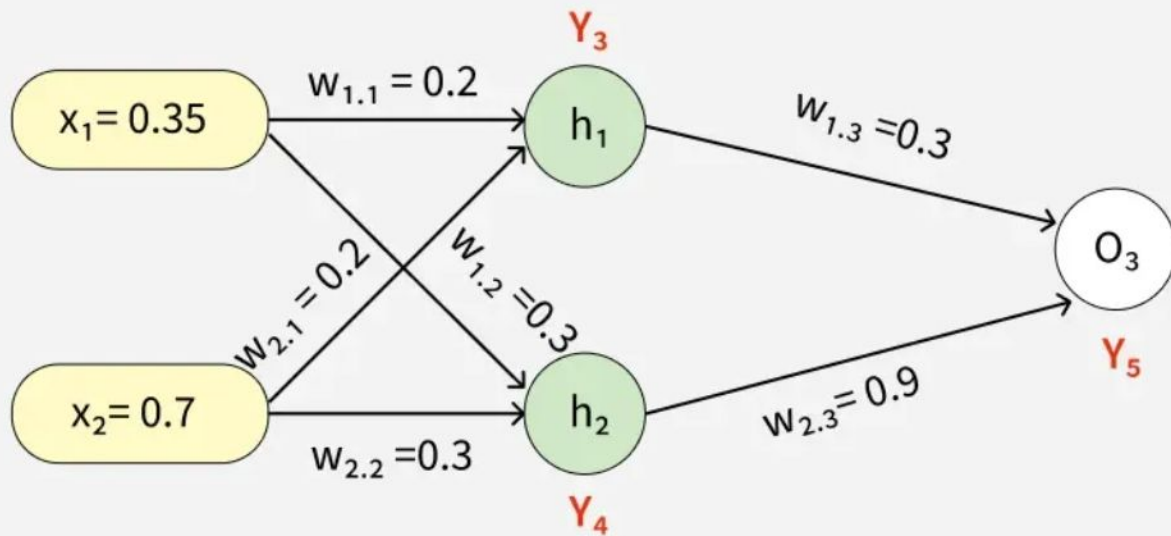
o (output): After applying the activation function to a , we get the output of the neuron:

$$o_j = \text{activation function}(a_j)$$

2. Sigmoid Function

The sigmoid function returns a value between 0 and 1, introducing non-linearity into the model.

$$y_j = \frac{1}{1+e^{-a_j}}$$



To find the outputs of y_3 , y_4 and y_5

3. Computing Outputs

At h_1 node

$$\begin{aligned}a_1 &= (w_{1,1}x_1) + (w_{2,1}x_2) \\&= (0.2 * 0.35) + (0.2 * 0.7) \\&= 0.21\end{aligned}$$

Once we calculated the a_1 value, we can now proceed to find the y_3 value:

$$\begin{aligned}y_j &= F(a_j) = \frac{1}{1+e^{-a_1}} \\y_3 &= F(0.21) = \frac{1}{1+e^{-0.21}} \\y_3 &= 0.56\end{aligned}$$

Similarly find the values of y_4 at h_2 and y_5 at O_3

$$\begin{aligned}a_2 &= (w_{1,2} * x_1) + (w_{2,2} * x_2) = (0.3 * 0.35) + (0.3 * 0.7) = 0.315 \\y_4 &= F(0.315) = \frac{1}{1+e^{-0.315}} \\a_3 &= (w_{1,3} * y_3) + (w_{2,3} * y_4) = (0.3 * 0.57) + (0.9 * 0.59) = 0.702 \\y_5 &= F(0.702) = \frac{1}{1+e^{-0.702}} = 0.67\end{aligned}$$

4. Error Calculation

Our actual output is 0.5 but we obtained 0.67. To calculate the error we can use the below formula:

$$Error_j = y_{target} - y_5$$

$$\Rightarrow 0.5 - 0.67 = -0.17$$

Using this error value we will be backpropagating.

$$x_1 = 0.05$$

$$x_2 = 0.10$$

$$y = 0.01$$

Weights from Input $\rightarrow$ Hidden:

$$w_1 = 0.15 \quad w_2 = 0.20$$

$$w_3 = 0.25 \quad w_4 = 0.30$$

Bias for hidden layer:

$$b_1 = 0.35$$

Weights from Hidden $\rightarrow$ Output:

$$w_5 = 0.40 \quad w_6 = 0.45$$

Bias for output layer:

$$b_2 = 0.60$$

Learning rate (η): 0.5

1. Hidden layer outputs

$$h1_input = x1*w1 + x2*w2 + b1$$

$$= 0.05*0.15 + 0.10*0.20 + 0.35 = 0.3775$$

$$h1_output = \text{sigmoid}(0.3775) \approx 0.5933$$

$$h2_input = x1*w3 + x2*w4 + b1$$

$$= 0.05*0.25 + 0.10*0.30 + 0.35 = 0.3925$$

$$h2_output = \text{sigmoid}(0.3925) \approx 0.5969$$

$$o_input = h1_output * w5 + h2_output * w6 + b2$$

$$= 0.5933 * 0.40 + 0.5969 * 0.45 + 0.60 \approx 1.1059$$

$$o_output = \text{sigmoid}(1.1059) \approx 0.7514$$

Backward Pass (Backpropagation)

Calculate Output Error

$$\text{Error} = 0.5 * (\text{target} - \text{output})^2$$

$$= 0.5 * (0.01 - 0.7514)^2 \approx 0.2748$$

Output layer delta

$$\delta_{\text{output}} = (\text{output} - \text{target}) * \text{sigmoid}'(\text{output_input})$$

$$= (0.7514 - 0.01) * \text{sigmoid}(1.1059) * (1 - \text{sigmoid}(1.1059))$$

$$= 0.7414 * 0.1868 \approx 0.1385$$

Gradients for w5, w6

$$\partial E / \partial w_5 = \delta_{\text{output}} * h1_{\text{output}} = 0.1385 * 0.5933 \approx 0.0822$$

$$\partial E / \partial w_6 = \delta_{\text{output}} * h2_{\text{output}} = 0.1385 * 0.5969 \approx 0.0826$$

Update w5, w6

$$w5_{\text{new}} = w5 - \eta * \partial E / \partial w_5 = 0.40 - 0.5 * 0.0822 \approx 0.3589$$

$$w6_{\text{new}} = w6 - \eta * \partial E / \partial w_6 = 0.45 - 0.5 * 0.0826 \approx 0.4087$$

Hidden layer deltas

We need to backpropagate error to hidden layer:

$$\delta_{h1} = \delta_{\text{output}} * w5 * \text{sigmoid}'(h1_input)$$

$$= 0.1385 * 0.40 * 0.5933 * (1 - 0.5933)$$

$$\approx 0.1385 * 0.40 * 0.241 \approx 0.0133$$

$$\delta_{h2} = \delta_{\text{output}} * w6 * \text{sigmoid}'(h2_input)$$

$$= 0.1385 * 0.45 * 0.5969 * (1 - 0.5969)$$

$$\approx 0.1385 * 0.45 * 0.2408 \approx 0.0150$$

Gradients for w1–w4

$$\partial E / \partial w_1 = \delta_{h1} * x_1 = 0.0133 * 0.05 \approx 0.0007$$

$$\partial E / \partial w_2 = \delta_{h1} * x_2 = 0.0133 * 0.10 \approx 0.0013$$

$$\partial E / \partial w_3 = \delta_{h2} * x_1 = 0.0150 * 0.05 \approx 0.0008$$

$$\partial E / \partial w_4 = \delta_{h2} * x_2 = 0.0150 * 0.10 \approx 0.0015$$

Update w1-w4

$$w1_new = w1 - \eta * \partial E / \partial w1 = 0.15 - 0.5 * 0.0007 \approx 0.1497$$

$$w2_new = w2 - 0.5 * 0.0013 \approx 0.1993$$

$$w3_new = w3 - 0.5 * 0.0008 \approx 0.2496$$

$$w4_new = w4 - 0.5 * 0.0015 \approx 0.2992$$